Micro Project: 3-DOF Robotic Manipulator

Kinematic Modeling, Forward Kinematics Formulation & 3D Simulation

Project Title:	3-DOF Revolute Robotic Manipulator (RRR)
Group Members:	1. [Student Name 1] - [Roll Number / ID] 2. [Student Name 2] - [Roll Number / ID] 3. [Student Name 3] - [Roll Number / ID]
Department:	[Department Name, e.g., Mechanical / Robotics / Mechatronics]
Institution:	[College / University Name]
Academic Year:	2025 - 2026

1. Algorithm (Pseudo-code)

The following step-by-step logic defines the geometric initialization, kinematic transformation pipeline, and 3D simulation of the 3-DOF articulated manipulator:

Step 1: Link Parameter Definition

Define physical link dimensions: Base height L1, Upper arm length L2, and Forearm length L3 in meters.

Step 2: DH Parameter Formulation

Assign Standard Denavit-Hartenberg parameters for each link:
• Link 1: d = L1, a = 0, alpha = +pi/2
• Link 2: d = 0, a = L2, alpha = 0
• Link 3: d = 0, a = L3, alpha = 0

Step 3: Robot Assembly

Instantiate RevoluteDH joint objects and assemble the serial-link DHRobot model.

Step 4: Input Joint Angles

Acquire joint variables q1, q2, and q3 in degrees and convert them into radians.

Step 5: Homogeneous Transformation Calculation

Compute individual link transformation matrices A1, A2, and A3, and evaluate the composite end-effector transformation matrix:

```
T_0_3 = A1 * A2 * A3
```

Step 6: End-Effector Position Extraction

Extract translational coordinates (X, Y, Z) from fourth column of T_0_3:

```
X = cos(q1) * [L2 * cos(q2) + L3 * cos(q2 + q3)]
Y = sin(q1) * [L2 * cos(q2) + L3 * cos(q2 + q3)]
Z = L1 + L2 * sin(q2) + L3 * sin(q2 + q3)
```

Step 7: 3D Kinematic Plotting

Pass joint angles to the robotics visualization environment to render links, joint axes, and coordinate frames in 3D Euclidean space, and save the plot.

2. Program

Core Python implementation utilizing Peter Corke's Robotics Toolbox for forward kinematics calculation and 3D simulation:

```
import numpy as np
import roboticstoolbox as rtb
import matplotlib.pyplot as plt

class Manipulator3DOF:
    def __init__(self, l1=0.5, l2=0.4, l3=0.3):
        self.l1 = l1
        self.l2 = l2
        self.l3 = l3

        self.link1 = rtb.RevoluteDH(d=self.l1, a=0.0, alpha=np.pi/2)
        self.link2 = rtb.RevoluteDH(d=0.0, a=self.l2, alpha=0.0)
        self.link3 = rtb.RevoluteDH(d=0.0, a=self.l3, alpha=0.0)

        self.robot = rtb.DHRobot([self.link1, self.link2, self.link3], name="3DOF_Manipulator")

    def forward_kinematics(self, q):
        return self.robot.fkine(q)

    def plot_arm(self, q, save_path=None):
        self.robot.plot(q, backend="pyplot", block=False)
        if save_path:
            plt.savefig(save_path, dpi=300)
        plt.close()

if __name__ == "__main__":
    arm = Manipulator3DOF(l1=0.5, l2=0.4, l3=0.3)

    test_angles = [
        [0.0, 0.0, 0.0],
        [30.0, 45.0, 30.0],
        [-45.0, 60.0, -30.0]
    ]

    for i, angles in enumerate(test_angles, 1):
        q_rad = np.deg2rad(angles)
        T = arm.forward_kinematics(q_rad)
        pos = T.t

        print(f"Test Configuration {i}:")
        print(f"Joint Angles (deg): q1 = {angles[0]}, q2 = {angles[1]}, q3 = {angles[2]}")
        print("Homogeneous Transformation Matrix:")
        print(np.round(T.A, 4))
        print(f"End-Effector Coordinates (m): X = {pos[0]:.4f}, Y = {pos[1]:.4f}, Z = {pos[2]:.4f}\n"
)

    arm.plot_arm(np.deg2rad([30.0, 45.0, 30.0]), save_path="manipulator_3d_pose.png")
```

3. Result

The script was executed for three distinct joint angle test configurations to evaluate the spatial displacement of the end-effector:

```
Test Configuration 1:
Joint Angles (deg): q1 = 0.0, q2 = 0.0, q3 = 0.0
Homogeneous Transformation Matrix:
[[ 1.  0.  0.  0.7]
 [ 0.  0. -1.  0. ]
 [ 0.  1.  0.  0.5]
 [ 0.  0.  0.  1. ]]
End-Effector Coordinates (m): X = 0.7000, Y = 0.0000, Z = 0.5000

Test Configuration 2:
Joint Angles (deg): q1 = 30.0, q2 = 45.0, q3 = 30.0
Homogeneous Transformation Matrix:
[[ 0.2241 -0.8365  0.5     0.3122]
 [ 0.1294 -0.483  -0.866   0.1802]
 [ 0.9659  0.2588  0.      1.0726]
 [ 0.      0.      0.      1.    ]]
End-Effector Coordinates (m): X = 0.3122, Y = 0.1802, Z = 1.0726

Test Configuration 3:
Joint Angles (deg): q1 = -45.0, q2 = 60.0, q3 = -30.0
Homogeneous Transformation Matrix:
[[ 0.6124 -0.3536 -0.7071  0.3251]
 [-0.6124  0.3536 -0.7071 -0.3251]
 [ 0.5     0.866   0.      0.9964]
 [ 0.      0.      0.      1.    ]]
End-Effector Coordinates (m): X = 0.3251, Y = -0.3251, Z = 0.9964
```

Config	Joint Angles (q1, q2, q3)	X (m)	Y (m)	Z (m)
1 (Home)	[0.0°, 0.0°, 0.0°]	0.7000	0.0000	0.5000
2 (Reach)	[30.0°, 45.0°, 30.0°]	0.3122	0.1802	1.0726
3 (Pick)	[-45.0°, 60.0°, -30.0°]	0.3251	-0.3251	0.9964

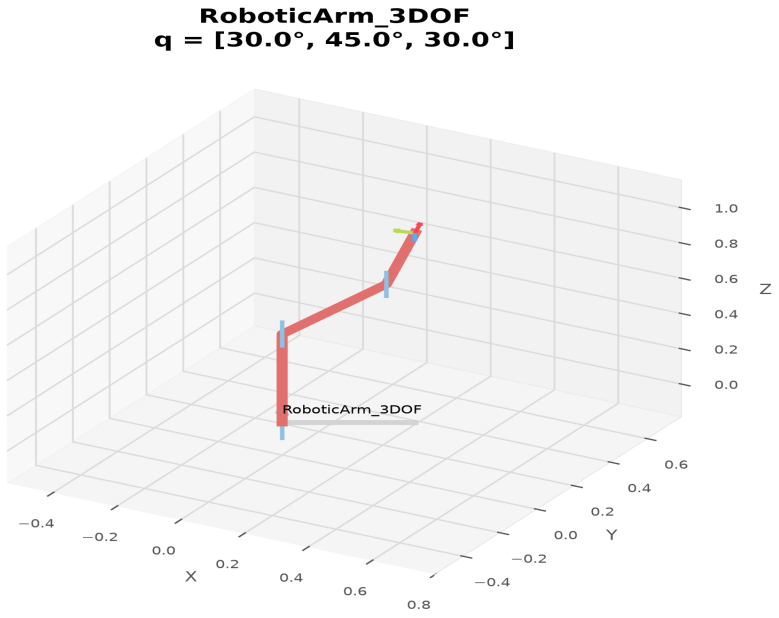


Figure 1: Generated 3D Robotic Manipulator Model at Joint Angles q = [30.0°, 45.0°, 30.0°]